

DOCUMENTACIÓN DEL PROYECTO

WhatsApp PyME Bot

Atención, catálogo, pedidos y pagos sobre WhatsApp Cloud API

Guía de uso y referencia técnica

Versión documentada API FastAPI multiempresa
Rama de referencia main
Responsable Jimmy Toapanta
Revisión 13/08/2026

Control del documento

Campo	Detalle
Proyecto	WhatsApp PyME Bot
Alcance	Manual de usuario y referencia técnica en un único documento
Versión documentada	API FastAPI multiempresa
Rama de referencia	main
Estado	Servicio validado localmente con 21 pruebas automatizadas
Público previsto	Pequeñas empresas, administradores del bot y mantenedores backend
Responsable	Jimmy Toapanta
Última revisión	13/08/2026

Cómo usar este manual

Empiece por el simulador: permite verificar catálogo, conversación y pedidos sin enviar mensajes reales. La parte técnica describe después el webhook, el aislamiento entre empresas, la base y las integraciones de Meta y OpenAI.

La guía separa el funcionamiento comprobado de las mejoras pendientes. Las dependencias externas y los puntos que requieren validación manual están señalados expresamente.

Contenido

Control del documento	2
Contenido	3
El proyecto en pocas palabras	4
Preparación y puesta en marcha	5
Operación paso a paso	6
Problemas frecuentes y recuperación	7
Arquitectura y componentes	8
Configuración, datos e integraciones	8
Flujos internos y decisiones de diseño	9
Desarrollo, pruebas y mantenimiento	10
Despliegue y continuidad	10
Límites conocidos y evolución	11
Entrega a otra persona	12

01 · CONTEXTO

El proyecto en pocas palabras

WhatsApp PyME Bot es un servicio FastAPI multiempresa conectado a WhatsApp Cloud API. Valida webhooks, evita duplicados, identifica la empresa, conserva el contexto de conversación, busca productos, interpreta intención, registra pedidos y notifica a la persona responsable.

Propósito y alcance

Ofrecer una primera atención consistente sin esconder al cliente que interactúa con una automatización. El bot resuelve consultas frecuentes y prepara pedidos; los casos ambiguos, pagos y excepciones deben poder escalar a una persona.

Qué resuelve

- Una instalación para varias empresas con configuración aislada.
- Validación de firma y deduplicación de eventos.
- Catálogo con búsqueda tolerante y conversación con historial.
- Pedidos, información de pago y aviso al responsable.
- Clasificador local y proveedor OpenAI opcional, API administrativa, simulador y documentación OpenAPI.

Estado actual

La aplicación usa SQLite en desarrollo y admite PostgreSQL en producción mediante Alembic. Los tokens de Meta se cifran. La rama main se validó con 21 pruebas. Existe una base local de prueba ignorada que no debe agregarse al commit de documentación.

Alcance de esta entrega

- Instalación o incorporación de la versión presente en la rama principal.
- Recorridos de uso, verificaciones y recuperación ante fallos habituales.
- Arquitectura, configuración, datos, integraciones, pruebas y despliegue.
- Límites conocidos y una ruta de evolución coherente con el estado del código.

Fuera de alcance

- Cobrar o confirmar un pago sin verificación por el canal correspondiente.
- Prometer disponibilidad o exactitud absoluta de un modelo externo.
- Responder desde producción sin HTTPS, firma de Meta, secretos rotados y política de privacidad.

02 · GUÍA DE USUARIO

Preparación y puesta en marcha

El recorrido más seguro empieza en el simulador. Configure una empresa, su catálogo y un responsable; después pruebe saludos, productos ambiguos, pedido y escalamiento antes de conectar un número real.

Requisitos previos

- Python 3.12 y entorno virtual.
- DATABASE_URL y secretos derivados de .env.example.
- Para producción, aplicación Meta, número de WhatsApp, token y app secret.
- HTTPS público para webhook y PostgreSQL recomendado en producción.

Instalación o incorporación

1. Crear y activar .venv.
2. Instalar requirements.txt.
3. Copiar .env.example a .env y generar secretos únicos.
4. Ejecutar alembic upgrade head.
5. Cargar datos de prueba.
6. Iniciar Uvicorn y usar /docs o el simulador.

```
cd /d \"RUTA\\whatsapp-pyme-bot\"
py -3.12 -m venv .venv
.venv\\Scripts\\activate.bat
pip install -r requirements.txt
copy .env.example .env
alembic upgrade head
python datos_prueba.py
uvicorn app.main:app --reload
```

Primera ejecución

1. Abrir /health y confirmar estado correcto.
2. Abrir /docs y localizar el simulador de mensaje.
3. Enviar un saludo ficticio con POST /test/simular-mensaje.
4. Consultar un producto, aclarar una coincidencia y formar un pedido.
5. Revisar la conversación, el pedido y la notificación al responsable en la base de desarrollo.

Señal de que todo quedó bien: un evento se procesa una sola vez, responde con el catálogo de la empresa correcta y conserva el pedido sin exponer claves en la respuesta o el log.

Operación paso a paso

Los recorridos siguientes están escritos para una jornada real de trabajo. Antes de confirmar una acción masiva, conviene validar primero un caso de prueba y revisar el resultado.

Simular una conversación

1. Iniciar Uvicorn.
2. Abrir /docs.
3. Enviar un JSON de prueba a /test/simular-mensaje.
4. Continuar con el mismo identificador para conservar contexto.
5. Revisar respuesta y estado creado.

Resultado esperado: el flujo completo puede probarse sin enviar un mensaje real.

Configurar una empresa

1. Acceder a la API administrativa con X-Admin-Key.
2. Crear empresa y parámetros públicos.
3. Registrar catálogo y responsable.
4. Configurar credenciales Meta cifradas.
5. Probar aislamiento con otra empresa ficticia.

Resultado esperado: cada mensaje encuentra una sola configuración y nunca consulta el catálogo vecino.

Recibir un webhook real

1. Configurar URL HTTPS y token de verificación.
2. Validar el reto inicial de Meta.
3. Recibir un evento firmado.
4. Comprobar firma y deduplicar por identificador.
5. Procesar y enviar la respuesta por Cloud API.

Resultado esperado: eventos repetidos no duplican mensajes ni pedidos.

Atender un pedido

1. El cliente consulta un producto.
2. El bot busca y aclara coincidencias ambiguas.
3. El cliente confirma ítems y cantidades.
4. El servicio crea el pedido y comunica instrucciones de pago configuradas.
5. Se notifica al responsable para revisión humana.

Resultado esperado: el pedido queda trazado y no se marca pagado por una afirmación no verificada.

Escalar a una persona

1. Detectar intención fuera de alcance, molestia o ambigüedad persistente.
2. Informar que una persona continuará.
3. Notificar al responsable con contexto mínimo.
4. Evitar seguir generando respuestas automáticas contradictorias.

5. Registrar la toma de control.

Resultado esperado: la conversación puede continuar de forma humana sin perder el contexto útil.

Lista de control operativa

- Comprobar /health y migraciones antes de recibir tráfico.
- Usar el simulador para saludo, producto ambiguo, pedido y escalamiento.
- Verificar que un evento repetido no cree dos respuestas ni dos pedidos.
- Confirmar que cada número se resuelve contra la empresa y el catálogo correctos.

Problemas frecuentes y recuperación

Si algo falla, conserve el mensaje exacto, la hora y la acción que lo produjo. Esa información suele ser más útil que reinstalar o borrar la configuración sin diagnóstico.

Síntoma	Causa probable	Qué revisar
/health falla	Base, migración o configuración incompleta	Revisar DATABASE_URL y alembic current/upgrade.
Meta rechaza webhook	Verify token, URL o HTTPS	Comparar WHATSAPP_VERIFY_TOKEN y configuración de Meta.
Firma inválida	META_APP_SECRET incorrecto o cuerpo alterado	Usar el cuerpo crudo y secreto correcto; nunca omitir en producción.
Responde empresa incorrecta	Mapeo de número o tenant	Revisar phone_number_id e aislamiento.
Duplica pedidos	Dedupe ausente o transacción incompleta	Comprobar ID de evento único y commit atómico.
OpenAI no responde	Clave, cuota o timeout	Mantener clasificador local y ruta de escalamiento.

Uso responsable, privacidad y seguridad

- Nunca confirmar contraseñas, tokens, claves API, perfiles del navegador ni archivos de configuración locales.
- Usar cuentas y permisos con el alcance mínimo necesario; retirar accesos cuando ya no se utilicen.
- Anonimizar correos, nombres, identificadores, órdenes y capturas antes de publicar evidencias en el portafolio.
- Revisar el historial completo de Git, no solo la última versión, antes de hacer público el repositorio.
- En producción, exigir validación de firma de Meta y HTTPS.
- Cifrar tokens Meta con ENCRYPTION_KEY, rotar ADMIN_SECRET_KEY y no registrar cuerpos completos con datos personales.

Preguntas habituales

¿Necesita IA externa?

No para todo. Existe un clasificador local; OpenAI es un fallback opcional para casos configurados.

¿SQLite sirve en producción?

Es adecuada para desarrollo. PostgreSQL ofrece mejor concurrencia y operación para un servicio desplegado.

¿El bot confirma pagos?

No por el solo mensaje del cliente. Debe existir verificación externa o revisión humana.

03 · REFERENCIA TÉCNICA

Arquitectura y componentes

FastAPI expone webhook, salud, simulador y administración. Un servicio de conversación coordina historial, catálogo, intención, IA, pedidos y mensajería. La persistencia aísla empresas y Alembic conserva el esquema.

Vista de arquitectura



La validación y deduplicación ocurren antes del caso de uso. Los adaptadores externos no deben decidir reglas de pedido.

Responsabilidad de los componentes

Componente	Responsabilidad	Tecnología o archivo
API	Webhook, salud, pruebas y administración	app/main.py y rutas
Conversación	Orquesta estado, intención y respuesta	Servicios de conversación
Catálogo	Búsqueda y aclaración de productos	Servicios/repositorios de catálogo
Pedidos/pagos	Persistencia y notificación	Dominio y servicios
Persistencia	SQLite/PostgreSQL y migraciones	SQLAlchemy, Alembic
Adaptadores	Meta Cloud API y OpenAI opcional	Clientes externos

Configuración, datos e integraciones

Parámetros importantes

Parámetro / recurso	Uso	Tratamiento recomendado
ENVIRONMENT	Desarrollo o producción	Producción activa controles estrictos.
DATABASE_URL	Conexión de persistencia	Usar secreto administrado.
ADMIN_SECRET_KEY	Protege administración	Valor largo, único y rotado.
ENCRYPTION_KEY	Cifra tokens Meta	Respaldar de forma segura; perderla impide descifrar.
WHATSAPP_VERIFY_TOKEN	Verificación inicial	Debe coincidir con Meta.
META_APP_SECRET	Firma de webhooks	Obligatorio en producción.
OPENAI_API_KEY / MODEL	Fallback opcional	Aplicar presupuesto, timeout y minimización.

Persistencia y archivos generados

- Las conversaciones y pedidos contienen datos personales: definir propósito, retención y acceso.
- test_whatsapp_bot.db es local e ignorada; no debe subirse.
- Alembic es la única ruta normal para cambiar el esquema.

- Los tokens Meta se almacenan cifrados, pero los logs también deben redactarse.

Integraciones externas

Integración	Función	Riesgo operativo
WhatsApp Cloud API	Recibir y enviar mensajes	Tokens, versiones API, plantillas y políticas Meta.
OpenAI opcional	Interpretación de casos configurados	Costo, latencia y respuesta inexacta.
PostgreSQL	Persistencia de producción	Disponibilidad, backups y migraciones.
Render u otro host	Ejecución pública	Secretos, arranque y escalado.
Responsable humano	Escalamiento de pedido	Disponibilidad y canal de notificación.

Flujos internos y decisiones de diseño

Entrada de webhook

1. Recibir cuerpo crudo y cabeceras.
2. Validar firma en producción.
3. Comprobar identificador contra deduplicación.
4. Resolver empresa por número.
5. Ejecutar conversación en una transacción.
6. Enviar respuesta y registrar resultado.

Responder rápido a Meta y separar trabajos largos evita reintentos que parecen duplicados.

Decisión conversacional

1. Recuperar historial mínimo.
2. Buscar catálogo y clasificar intención localmente.
3. Pedir aclaración si hay varias coincidencias.
4. Usar IA externa solo si está configurada y aporta valor.
5. Crear pedido o escalar según la intención.

La IA no debe saltarse reglas de catálogo, precio, pago o aislamiento entre empresas.

Criterios de diseño

- Separar configuración y credenciales del código que se versiona.
- Mantener una entrada clara para operar y otra para diagnóstico o pruebas.
- Preferir fallos explícitos a continuar con datos parciales o inventados.
- Conservar trazabilidad suficiente para reconstruir qué ocurrió sin exponer información sensible.

Desarrollo, pruebas y mantenimiento

Entorno de desarrollo

- Python 3.12, entorno virtual, Git y una base de datos aislada para desarrollo.
- Variables locales derivadas de .env.example, sin reutilizar secretos de producción.
- Cliente HTTP o interfaz OpenAPI para repetir solicitudes controladas.

```
pytest
alembic current
alembic upgrade head
uvicorn app.main:app --reload
```

Estrategia de pruebas

- Ejecutar las 21 pruebas automatizadas de la versión validada.
- Cubrir firma inválida, evento duplicado y aislamiento entre empresas.
- Probar fallback sin OpenAI y timeout del proveedor.
- Validar migraciones desde base vacía y desde la revisión anterior.

Registro, diagnóstico y soporte

- Conservar el mensaje completo y la hora del evento; evitar capturas que muestren credenciales o datos personales.
- Distinguir un error local de un fallo del servicio externo comprobando conectividad, permisos y cuotas.
- Repetir el caso con una entrada pequeña y controlada antes de cambiar dependencias o borrar datos.

Mantenimiento recomendado

- Revisar dependencias y servicios externos de manera periódica, probando la actualización en una rama separada.
- Actualizar este manual cuando cambien comandos, configuración, estructura de datos o flujo de usuario.
- Mantener ejemplos sin secretos y una prueba mínima que permita validar la instalación.
- Etiquetar versiones estables y documentar las alternativas como ramas, no como reemplazos ambiguos.

Despliegue y continuidad

Ruta de despliegue

1. Crear la base de datos y los secretos en el entorno de destino.
2. Instalar dependencias y ejecutar migraciones antes de iniciar el servicio.
3. Levantar una instancia de prueba y comprobar salud, autenticación y persistencia.
4. Configurar el webhook o punto de entrada externo con HTTPS.
5. Observar una conversación de prueba antes de habilitar tráfico real.

```
alembic upgrade head
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Respaldo y recuperación

- Respaldar solo datos necesarios y cifrar cualquier copia que contenga información personal u operativa.
- Probar la restauración en una ubicación distinta antes de confiar en el respaldo.
- Conservar una copia de la configuración sin secretos y registrar por separado dónde se administran las credenciales.

Criterios de aceptación

- Una instalación limpia puede completar el recorrido principal descrito en este manual.
- Los errores de credenciales, permisos o red se informan sin filtrar secretos.
- Los archivos temporales y los datos generados permanecen fuera del repositorio.
- La persona responsable puede identificar versión, configuración y evidencia de la última ejecución correcta.

Límites conocidos y evolución

Límites de la versión documentada

- Depende de políticas y versiones de Meta.
- Un modelo generativo puede equivocarse y añade costo/latencia.
- La notificación humana no garantiza respuesta inmediata.
- El tratamiento de datos requiere política de privacidad y controles fuera del código.

Hoja de ruta sugerida

Prioridad	Mejora	Resultado esperado
Alta	Cola asíncrona e idempotencia transaccional	Responder al webhook rápido y procesar reintentos con seguridad.
Alta	Pruebas de aislamiento multiempresa	Prevenir filtración cruzada.
Media	Panel de conversación y toma humana	Gestionar escalamiento de forma visible.
Media	Métricas redactadas de intención y resolución	Mejorar sin almacenar mensajes innecesarios.

Versiones y ramas

Rama / versión	Papel en el historial	Recomendación
main · FastAPI multiempresa	Línea estable	Publicar y desplegar desde esta rama.
Modo simulador	Entrada local sin WhatsApp real	Usar para desarrollo y demostraciones.

Entrega a otra persona

Este apartado resume lo mínimo que debe conocer quien reciba el mantenimiento. La intención es que el proyecto pueda retomarse sin depender de la memoria del autor.

Paquete de traspaso

- Repositorio, rama y etiqueta exacta que se consideran estables.
- Lista de servicios externos, responsables de las cuentas y procedimiento para rotar accesos.
- Configuración de ejemplo sin secretos y ubicación de los datos persistentes.
- Resultado de la última prueba, limitaciones abiertas y próximos cambios recomendados.
- Un caso de prueba anonimizado que permita recorrer el flujo de principio a fin.

Glosario breve

Término	Significado en este proyecto
Webhook	Solicitud que Meta envía cuando ocurre un evento.
Tenant	Empresa aislada dentro de la misma aplicación.
Deduplicación	Evitar procesar dos veces el mismo evento.
Alembic	Historial de cambios de base de datos.
Fallback	Alternativa usada cuando el clasificador principal no resuelve.

Cierre

El bot debe facilitar una conversación, no encerrarla. Firma, deduplicación, aislamiento y una salida clara hacia atención humana son requisitos operativos, no detalles internos.

Documento mantenido junto al código fuente. Las decisiones operativas que cambien el comportamiento deben reflejarse tanto en el código como en esta guía.